CSE 213 — Software Engineering

Process Models and Agile Development

Lesson notes · Exam approach · TF practice with solutions · Quick notes (IJ Mam's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — process-model and agile questions
Lesson 12 — Generic Process Model: Framework Activities and Process Flows — pp. 2–16
Lesson 13 — Prescriptive Models: Waterfall, Incremental, Prototyping, Spiral (+ V, RAD, UP) — pp. 15–42
Lesson 14 — Agile Principles and Extreme Programming — pp. 43–63
Lesson 15 — Scrum and Kanban — pp. 64–84
Quick Notes — Process Models and Agile — revision + key diagrams

> **NOTE** Slide numbers refer to the PDF page numbers of the deck named under each lesson title. TF page numbers refer to 'CSE 213 All Past Papers.pdf' (70 pages); the newest paper (22 batch, labelled 2024-25 here) is a separate file, cited as '22 batch p.N'. Old course code: CSE 307. Page numbers refer to '4 - Process Models and Agile Development .pdf' (84 pages, 'Prepared by AYAS'). V model, RAD and the Unified Process are not in the slides; they are given briefly from Pressman (marked 'textbook') because older papers ask them.

Exam Map — What the TF Papers Ask

Session (paper)	Questions in this note file
2024-25 (CSE 213)	Q1(c) Mental-health companion app: is the incremental model suitable? Propose alternatives (10) [L13]
2023-24 (CSE 213)	Q8(a) Scenario: incremental vs agile — which and why (8) [L14]
2022-23 (CSE 307)	Q6(a) Core activities of the generic process model; waterfall + advantages/disadvantages (10) [L12] · Q6(b) Kanban: workload per card and lead time (10) [L15] · Q6(c) Sprint activities in Scrum; the two backlogs (15) [L15]
2021-22 (CSE 307)	Q5(b) Steps of a sprint in Scrum (20) [L15] · Q6(a) How incremental and evolutionary models address waterfall shortcomings (10) [L13] · Q7(a) Reasons to shift from traditional to agile (10) [L14] · Q8(b) Kanban visualisation: cards and board (15) [L15]
2018-19 (CSE 307)	Q5(b) Development cost vs schedule progress graph (agile) (10) [L14] · Q5(c) Framework activity; five PSP framework activities (13) [L12] · Q7(a) Three phases of ASD; XP phases diagram (16) [L14] · Q8(a) V model (7) [L13] · Q8(b) When incremental model; four problems of evolutionary models (10) [L13]
2017-18 (CSE 307)	Q2(a) Problems of the waterfall model and which models solve them (7) [L13] · Q2(b) Prototyping to elicit an SRS for a government organisation (7) [L13] · Q2(d) Is pair programming more efficient? (7) [L14] · Q2(e) Scrum meeting minutes for a digital library (7) [L15]
2016-17 (CSE 307)	Q5(b) Spiral model; arguments for and against (15) [L13] · Q6(a) Short notes: sprint, backlog, pair programming, refactoring, spike solution (15) [L15] · Q6(b) How XP follows the agility principles (15) [L14]
2014-15 (CSE 307)	Q4(a) Why framework activities do not suit every project (6⅔) [L12] · Q4(b) Incremental vs spiral (10) [L13] · Q4(c) Unified process; concurrency and evolution (10) [L13] · Q4(d) Distinguishing features of Scrum (10) [L15] · Q5(b) Disadvantages of prototyping (10) [L13]
2013-14 (CSE 307)	Q4(c) Framework and umbrella activities (8) [L12] · Q4(d) Suitable process model for three cases (9) [L13] · Q5(c) Discovery prototyping (5) [L13] · Q6(a) What is an agile method? (6⅔) [L14]
2012-13 (CSE 307)	Q1(b) 'Waterfall may lead to blocking states' — agree? (7) [L13] · Q1(d) Phases of the Unified Process (15) [L13] · Q2(c) RAD model; why a high-speed waterfall (12) [L13] · Q2(d) Project velocity in XP (6⅔) [L14]
2011-12 (CSE 307)	Q5(b) Agile; stages of extreme programming (18⅔) [L14] · Q5(c) Scrum vs plan-based project management (18) [L15] · Q7(c) Estimation techniques at stages of agile development (11⅔) [L14] · Q8(a) Impatient client who keeps changing requirements: which process model? (10) [L13] · Q8(b) Four phases of RUP; four differences from waterfall (12) [L13]
2010-11 (CSE 307)	Q6(a) Waterfall vs iterative models (6) [L13]

EXAM FOCUS Asked in every paper as 7–20-mark theory: scenario → which process model and why (2024-25, 2023-24, older), waterfall and generic activities (2022-23), Scrum sprint steps and backlogs (2022-23, 2021-22 — 20 marks), Kanban card workload (2022-23, 2021-22), reasons for agile (2021-22). Learn the advantage/disadvantage lists word for word, the sprint flow and the 'model choice' recipe.

Lesson 12 — Generic Process Model: Framework Activities and Process Flows

Pages 2–16

Software is embodied knowledge that is initially scattered and incomplete; a step-by-step process brings that knowledge into the software (p. 3). A **process model** divides software-development work into distinct phases and gives a road map (p. 4).

Framework activity	What happens (pp. 6–8)
1 Communication	project team communicates with the client; requirements gathered
2 Planning	schedule the development and estimate the cost
3 Modeling	analyse requirements; design algorithms, flowcharts, components
4 Construction	implement and test the components
5 Deployment	deliver to the client; feedback used to improve quality

Process flow	Meaning (pp. 9–14)
Linear	the five activities once, in sequence
Iterative	repeat one or more activities before moving on
Evolutionary	activities in a circle; each cycle (incremental release) gives a more complete version
Parallel	one or more activities run in parallel with others (e.g. modeling of one part with construction of another)

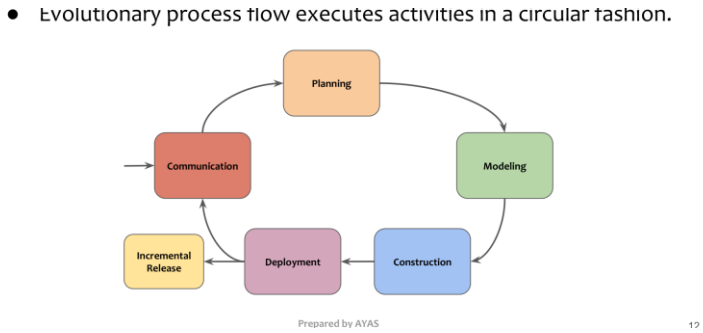


Figure 12.1 — Evolutionary process flow (p. 12)

> **NOTE** Textbook background: **umbrella activities** run across all framework activities — project tracking and control, risk management, quality assurance, technical reviews, measurement, configuration management, reusability management, work-product preparation. **PSP framework activities**: planning, high-level design, high-level design review, development, postmortem.

Practice — Lesson 12

(a) Actual — 2022-23 Q6(a). (b) Actual — 2013-14 Q4(c). (c) Actual — 2018-19 Q5(c). (d) Actual — 2014-15 Q4(a).

(a) Core activities of the generic process model; describe waterfall with advantages and disadvantages. (10)

(b) Framework and umbrella activities. (8)

(c) What does framework activity mean? Name five PSP framework activities. (3+10)

(d) Why are framework activities not applicable to every project in the same way? (6⅔)

✓ Solution

(a) Five activities from the first table, then waterfall (Lesson 13 §1) — the five activities run once in order; advantages: simple, understandable, manageable; disadvantages: all requirements up front, testing and client evaluation at the end (high risk), no iteration.

(b) Framework activities = the five phases that produce the product. Umbrella activities = NOTE list; they support every phase and manage progress, quality, change and risk.

(c) A framework activity is one of the generic steps applicable to all software projects regardless of size. PSP: planning, high-level design, high-level design review, development, postmortem.

(d) The activities are always present but their depth, order and repetition depend on the project: a small web page needs light planning, a safety-critical system heavy modeling and reviews; agile projects interleave them in short iterations. The process flow (linear/iterative/evolutionary/parallel) adapts them.

Lesson 13 — Prescriptive Models: Waterfall, Incremental, Prototyping, Spiral (+ V, RAD, UP)

Pages 15–42

Model	How it works	Advantages	Disadvantages	Use when
Waterfall (pp. 17–22)	communication → planning → modeling → construction → deployment, once, sequentially	simple to use; understandable workflow; easily manageable as requirements are known	client must give all requirements up front; testing and client evaluation only at the end → high risk; no iteration	small projects, well-defined stable requirements
Incremental (pp. 23–28)	core product first, then increments adding features; feedback after each	flexible to change; modifications throughout; errors mitigated by client review; early working product; easy testing; low initial cost	hard to split into increments; total cost high; complete planning needed up front; refining requirements may hurt architecture	client wants primitive functionality quickly
Prototyping (pp. 31–35)	build prototypes from known requirements; client evaluates; repeat until acceptable	active client involvement; early error detection; detailed requirements not needed to start	can slow the process; frequent change adds complexity; prototypes may be scrapped	vague requirements, unclear module behaviour
Spiral (pp. 36–40)	series of versions; each loop includes risk analysis (planning, risk analysis, engineering, evaluation)	rigorous risk handling; changes possible late	costly and complicated; needs risk-assessment expertise	large, high-risk, uncertain projects

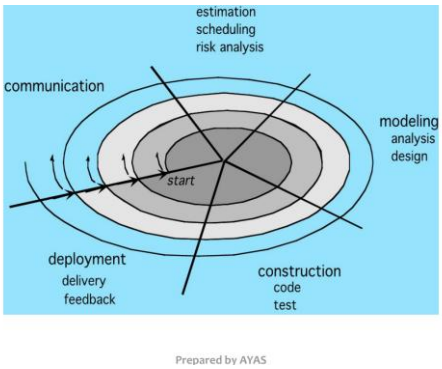


Figure 13.1 — Spiral model (p. 36)

Evolutionary (pp. 29–30): suits changing requirements when a complete product cannot be delivered by the deadline but a limited version can; unlike incremental (requirements fixed up front, refined per increment), requirements evolve. Food for thought (pp. 41–42): prescriptive models bring order, but some chaos aids creativity — leads to agile.

> **NOTE** Textbook models for older questions. **V model**: waterfall with a test level paired to each phase — requirements ↔ acceptance test, system design ↔ system test, architecture ↔ integration test, module design ↔ unit test, coding at the bottom. **RAD**: 60–90-day cycles, component reuse and parallel teams; a 'high-speed waterfall' because the same phases are compressed with automation and reuse. **Unified Process** (RUP): phases inception, elaboration, construction, transition (+ production); use-case driven, architecture-centric, iterative and incremental; disciplines (workflows) run concurrently in each phase, so the product evolves.

Practice — Lesson 13

(a) Actual — 2024-25 Q1(c). (b) Actual — 2021-22 Q6(a) / 2017-18 Q2(a). (c) Actual — 2016-17 Q5(b). (d) Actual — 2013-14 Q4(d) / 2011-12 Q8(a). (e) Actual — 2012-13 Q1(b). (f) Actual — 2011-12 Q8(b) / 2012-13 Q1(d). (g) Actual — 2018-19 Q8(a)–(b).

(a) 'Personalized Mental Health Companion' app (track mental states, AI exercises, virtual companion); only high-level requirements; the team chooses incremental development to get feedback after each part. Is incremental suitable? Propose alternatives if more appropriate. (10)

(b) How do incremental and evolutionary models address waterfall's shortcomings? / Problems of waterfall and models that solve them. (10 / 7)

(c) Describe the spiral model; arguments for and against it. (15)

(d) Suitable process model: (i) innovative scientific software with unknown issues (ii) quick stock-brokerage solution for a new instrument (iii) large software with very unstable users; impatient, result-oriented client who keeps changing requirements. (9 / 10)

(e) 'Waterfall may lead to blocking states' — agree? (7)

(f) Four phases of RUP and four differences from waterfall. (12 / 15)

(g) V model; when to use incremental; four problems of evolutionary models. (7, 2+8)

✓ Solution

(a) Partly suitable. For: early working core (mood tracking) gives feedback; requirements are high-level so increments let them be refined; the client sees progress. Against: AI exercises and a virtual companion are uncertain and **high-risk** (clinical safety, data privacy, model behaviour); incremental needs a fairly complete plan and architecture up front and refining requirements each increment can break the architecture. Better: **Spiral** (risk analysis each loop — privacy, safety, AI accuracy) combined with **prototyping** of the companion UI, or an **agile (Scrum)** process with short sprints, user testing and a risk backlog. Recommended: spiral/agile hybrid — build a prototype of the AI companion in early loops, then deliver features in sprints.

(b) Waterfall shortcomings: requirements fixed early, late testing, late client feedback, no iteration, nothing usable until the end. Incremental: a working core early, feedback after each increment, changes accommodated in later increments, errors found earlier, lower initial cost. Evolutionary (prototyping/spiral): requirements can evolve; prototypes clarify vague needs; spiral adds explicit risk analysis per loop.

(c) Description from the table and figure; for: risk-driven, accommodates change, early prototypes, suits large projects; against: costly and complex, depends on risk-assessment expertise, hard to convince clients the evolution is controllable, not for small projects.

(d) (i) Spiral (or prototyping) — innovation and unknown issues need risk analysis and experiments. (ii) RAD / incremental or Scrum — quick delivery of core functionality, requirements fairly clear. (iii) Prototyping / agile — unstable users need to see and react to working versions. Impatient client who keeps changing requirements: an **agile model (Scrum or XP)** — fixed-length sprints keep control (changes go into the backlog and are prioritised for the next sprint), the client sees working software every 2–4 weeks.

(e) Agree. In waterfall each phase waits for the previous one to be completed and signed off; team members who finish early (e.g. testers) sit idle until construction ends, and a question in one phase blocks the next — 'blocking states'. Iterative/agile models avoid this by overlapping activities.

(f) Inception (business case, scope, key use cases), elaboration (architecture baseline, most use cases, risks), construction (build and test increments), transition (beta, deployment, training). Differences: RUP is iterative and incremental (waterfall single pass); disciplines overlap in every phase (waterfall: one activity per phase); risks attacked early through architecture; working executable at the end of each iteration; use-case driven; changes accommodated between iterations.

(g) V model — NOTE above. Incremental when the client needs core functionality quickly, requirements are mostly known, staffing is limited, or technical risks can be deferred. Evolutionary problems: process not visible (no regular deliverables); systems poorly structured by continual change; hard to plan cost/time (unknown number of cycles); special skills needed; prototype may be mistaken for the product.

Lesson 14 — Agile Principles and Extreme Programming

Pages 43–63

Why agile (p. 44): earlier models are slow, business is fast and changing. Agile = 'moving quickly' — rapid, incremental delivery; delivery over analysis and design; continuous communication with the client (pp. 46–49). Agile processes discourage detailed analysis, planning and documentation and are flexible to change in team and product. Plans are short-lived: some features persist, some change, some are dropped; focus is on construction (pp. 50–51).

The 12 agile principles (Agile Manifesto 2001, pp. 52–55)

1. Satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late, for the client's competitive advantage.
3. Deliver working software frequently — weeks to a couple of months, prefer shorter.
4. Client and developers work together regularly throughout the project.
5. Build projects around motivated individuals; give them environment, support and trust.
6. Face-to-face conversation is the most efficient way to convey information.
7. Working software is the primary measure of progress.
8. Sustainable development: a constant pace indefinitely.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity — the art of maximising the amount of work not done — is essential.
11. The best architectures, requirements and designs emerge from self-organising teams.
12. At regular intervals the team reflects on how to become more effective and adjusts.

Human factors (p. 56): competent, skilful, knowledgeable, committed, collaborative, decisive, faithful, respectful, self-organised people.

Extreme Programming (pp. 58–63)

- Kent Beck: take good practices to extreme levels from day one; lightweight — the simplest thing that works.
- Values: communication with clients, simplicity, feedback, respect (and courage); customer satisfaction first.
- Whole-team problem solving; **pair programming** — two developers at one computer design, code and test user stories.
- Advantages: simple code easy to modify; testing from day one; energising environment. Disadvantages: design may be neglected; little documentation and monitoring → repeated errors.
- Planning/feedback loops (p. 63): release plan (months) → iteration plan (weeks) → acceptance test (days) → stand-up (one day) → pair negotiation (hours) → unit test (minutes) → pair programming (seconds).

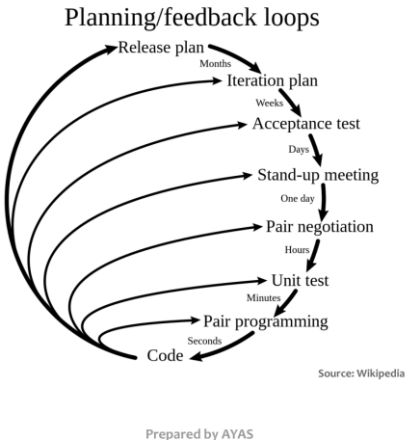


Figure 14.1 — XP planning/feedback loops (p. 63)

> **NOTE** Textbook XP activities: planning (user stories, story points, **project velocity** = stories (points) completed per iteration, used to plan the next release), design (KISS, CRC cards, **spike solutions** = quick prototypes for a hard problem), coding (unit tests first, pair programming, continuous integration), testing (daily unit tests, customer acceptance tests). ASD phases: speculation, collaboration, learning.

Practice — Lesson 14

(a) Actual — 2021-22 Q7(a). (b) Actual — 2023-24 Q8(a). (c) Actual — 2016-17 Q6(b). (d) Actual — 2018-19 Q5(b). (e) Actual — 2017-18 Q2(d). (f) Actual — 2012-13 Q2(d).

(a) Reasons to shift from traditional to agile development. (10)

(b) A scenario chooses between incremental and agile development — justify the better choice. (8)

(c) Explain how XP follows the agility principles. (15)

(d) Draw development cost vs schedule progress for agile development. (10)

(e) Is pair programming more efficient? (7)

(f) Project velocity in XP with an example. (6⅔)

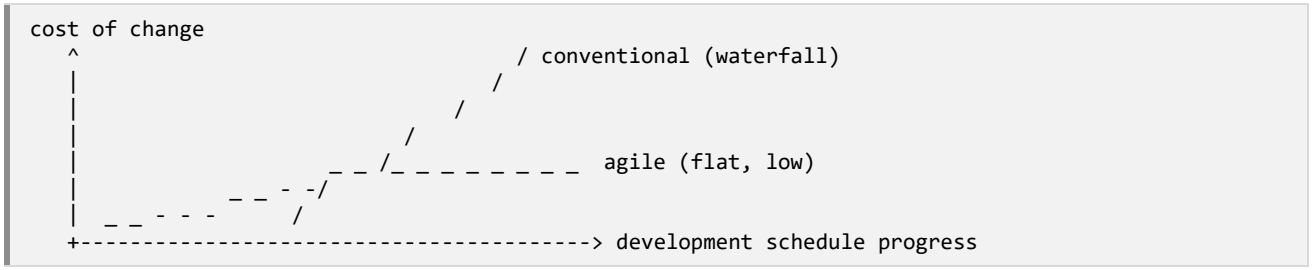
✓ Solution

(a) Requirements change fast and cannot be frozen; markets need early releases; waterfall gives late feedback and high risk; heavy documentation slows teams; agile delivers working software every few weeks, welcomes change, involves the client continuously, reduces risk through frequent feedback, and motivates self-organising teams (cite principles 1, 2, 3, 4, 7).

(b) Choose agile when requirements are unclear or changing and the client can be involved continuously; choose incremental when requirements are known and stable, a complete plan/architecture is possible, and contracts need fixed scope. State the scenario's facts against these two lists.

(c) Map XP practices to principles: small releases (1, 3); on-site customer and user stories (4, 6); welcome change via short iterations and refactoring (2); test-first and continuous integration (7, 9); simple design, YAGNI (10); 40-hour week (8); pair programming and collective ownership (5, 11); iteration retrospective (12).

(d) In agile the cost of change stays almost flat as the schedule progresses, while in conventional development it rises steeply:



(e) Usually yes for quality: continuous review catches defects early, knowledge spreads, design is better, less rework; studies show about 15 % more effort but ~15 % fewer defects. Less efficient for trivial tasks or mismatched pairs. Overall efficient when rework and maintenance are counted.

(f) Velocity = story points completed in an iteration. Example: iterations completed 20, 24, 22 points → average 22; the release backlog has 132 points → about 6 iterations; if velocity drops, re-plan or reduce scope.

Lesson 15 — Scrum and Kanban

Pages 64–84

Scrum (pp. 64–72)

Named after the rugby formation; a lightweight, iterative, incremental agile process — simple, gradual, frequent delivery; widely used.

Term	Meaning
Sprint	unit cycle of development, 2 weeks to 1 month
Product backlog	all client requirements of the product
Sprint backlog	requirements selected for the current sprint
Daily scrum	daily synchronisation meeting on plans and progress
Scrum master	usually the project manager (facilitator)
Product owner	the client (owns and prioritises the product backlog)
Tickets / scrum board	subtasks and the board that tracks them

```
Product backlog --(sprint planning: choose items)--> Sprint backlog
      ^                                                    |
      |  unfinished items go back                          v
      +-----------------------------------------  SPRINT (2-4 weeks)
                                                 planning -> implementation (daily scrum) ->
                                                 product review -> process review (retrospective)
                                                           |
                                                           v
                                        Potentially shippable increment -> client feedback
```

Advantages: less hierarchy, freedom; adaptability; products deliverable sooner; more client involvement; retrospective reduces mistakes. Disadvantage: works best with small teams.

Kanban (pp. 73–83)

看板 = signboard. Heavy visualisation so everyone is always up to date. Tasks split into subtasks, each on a Kanban card with the user story; cards move on a board (To-Do → In Progress → Done). **Pull system**: a free member pulls the next card. Agile coach, daily stand-ups and retrospectives exist, but there are **no sprints** — continuous flow. Advantages: leadership from everyone, adapts to the team, workload per card adjusted to keep a decent lead time. Disadvantage: no fixed time-frame → possible delay.

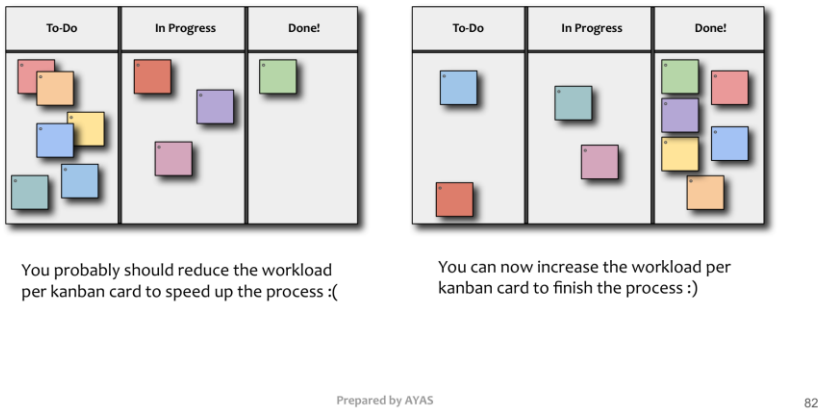


Figure 15.1 — Adjusting workload per Kanban card (p. 81)

Practice — Lesson 15

(a) Actual — 2022-23 Q6(c) / 2021-22 Q5(b). (b) Actual — 2022-23 Q6(b) / 2021-22 Q8(b). (c) Actual — 2016-17 Q6(a). (d) Actual — 2017-18 Q2(e). (e) Actual — 2014-15 Q4(d) / 2011-12 Q5(c).

(a) Activities in a Scrum sprint; the two kinds of backlog. (15 / 20)

(b) Kanban: how workload per card affects lead time; visualisation with cards and board. (10 / 15)

(c) Short notes: sprint, backlog, pair programming, code refactoring, spike solution. (3×5)

(d) Write Scrum meeting minutes for a digital library project. (7)

(e) Distinguishing features of Scrum; Scrum vs plan-based management (people allocation, cost estimation, team cohesion, membership changes). (10 / 18)

✓ Solution

(a) Product owner orders the product backlog → sprint planning selects items into the sprint backlog and breaks them into tickets → implementation with daily scrums (what I did, what I will do, blockers) → product review (demo to the client) → retrospective (process review) → increment delivered; unfinished items return to the product backlog for the next sprint. Product backlog = all requirements; sprint backlog = this sprint's subset.

(b) Lead time = time for a card from To-Do to Done. If cards pile up in In Progress, each card is too big — reduce the work per card (split it) so cards flow and lead time falls; if cards fly through with idle capacity, increase the work per card (p. 81). Board: columns To-Do / In Progress / Done (often with WIP limits), cards show the subtask, the user story, owner and due date; everyone sees status at a glance.

(c) Sprint and backlog — table. Pair programming — two developers, one computer: driver types, navigator reviews; roles swap. Refactoring — improving internal structure without changing behaviour (Refactoring notes). Spike solution — a small throw-away prototype to explore a risky technical problem and estimate it.

(d) Daily scrum, Digital Library, Sprint 3, Day 4 — Present: PO Rahim (observer), SM Nadia, devs A, B, C. A: done 'search by author'; today 'search by ISBN'; blocker: none. B: done borrow API; today due-date reminder; blocker: SMS gateway key pending (SM to follow up). C: done catalogue UI; today pagination; blocker: none. Burndown: 18 of 30 points left, on track. Action items: SM gets SMS key by 2 pm. Next scrum: 10 am tomorrow.

(e) Features: sprints, roles (PO, SM, team), backlogs, daily scrum, review, retrospective, time-boxing, self-organisation. vs plan-based: people allocation — team self-assigns tasks per sprint vs manager assigns from a full plan; cost estimation — per sprint from velocity and story points, refined continuously vs up-front estimate of the whole project; team cohesion — stable small cross-functional team with daily contact vs functional silos; membership change — knowledge shared through daily scrum and collective ownership makes replacement easier, while plan-based relies on documentation.

Quick Notes — Process Models and Agile

Revision points for Lessons 12–15.

L12 · Generic

- Communication, planning, modeling, construction, deployment.
- Flows: linear, iterative, evolutionary, parallel. Umbrella activities across all.

L13 · Prescriptive

- Waterfall (small, stable), incremental (core first), prototyping (vague), spiral (risk).
- Know 3 advantages + 3 disadvantages each. V, RAD, UP from the textbook note.

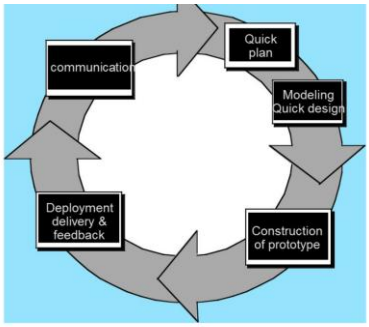
L14 · Agile & XP

- 12 principles; working software = progress; welcome change.
- XP: pair programming, test-first, small releases, simple design; velocity; spike.

L15 · Scrum & Kanban

- Sprint 2–4 weeks; product vs sprint backlog; daily scrum; review; retrospective.
- Kanban: board, cards, pull, no sprints; adjust workload per card for lead time.

Key Diagrams to Reproduce



Prototyping model (p. 31)